# UNIT – 2

## Regular Languages:

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$

### Regular expressions

(1Q) write a R.E for set of all strings length 2 over $\{a, b\}$

Sol: Set $= \{aa, ba, ab, bb\}$ → It is finite set

$RE = aa + bb + ba + ab \Rightarrow (a+b)\cdot(a+b)$

$= a[a+b] + b[a+b] \Rightarrow (a+b).(a+b)$

(2Q) RE for set of all strings of length atleast 2.

Sol: set $= \{aa, ba, ab, bb, aaa, baa \ldots\}$

$RE = (a+b)(a+b)(a+b)^*$ [∵ * means 0 (or) more]

Generate abab

abab

(3Q) set of all strings starting with 'a'

Sol: set $= \{a, ab, aa, aba \ldots\}$

$RE = a(a+b)^*$

(4Q) set of all strings ending with a

Sol: Set $= \{a, aa, ba, aab \ldots\}$

$RE = (a+b)^*a$

(5) set of all strings containing a on over {a,b}

Sol: set = {a, ba ab, bab, abb, ...

$$RE = (a+b)^* a (a+b)^*$$

(6) set of all strings that generate even length string over {a,b}

Sol: set = {ε, aa, ba, ab, bb, aaaa, baab, bbab, a...

Step 1: Two length string = aa + ba + ab + bb

$= a(a+b) + b(a+b)$

$= (a+b)(a+b)$

Repeat 2 length string.

$$RE = [(a+b)(a+b)]^*$$

If you want 4th length string from RE

Take * value is "2" so

$[(a+b)(a+b)]^2$

$(a+b)(a+b)(a+b)(a+b)$ If you want baba

b a b a = baba

(7) set of all strings that generate odd length string over $\{a,b\}$

Sol: $RE = [(a+b)(a+b)]^* (a+b)$

(or)

$= (a+b)[(a+b)(a+b)]^*$

(8) set of all strings that ~~generate~~ whose length is divisible by 3.

Sol- set $= \{aaa, baa, abb \ldots\}$

Step 1: only length '3'

$= (a+b)(b+a)(a+b) \cdot (a+b)(b+a)(a+b) \cdot$

$(a+b)(a+b)(a+b) \ldots$

$= [(a+b)(a+b)(a+b)]^*$

(9) Define RE to denote any no of '0' are followed by any no of 1 followed by any no of 2's.

Sol- set $= \{000111222, 012, 001122 \ldots$

~~RE~~ $= 0^* 1^* 2^*$

## Identity Rules of RE

(i) $\phi + R = R + \phi = R$

(ii) $\phi R = R \cdot \phi = \phi$

(iii) $\epsilon \cdot R = R \cdot \epsilon = R$

(v) $\epsilon^* = \epsilon$ (vi) $\phi^* = \epsilon$

(7) $\epsilon + RR^* = R^*R + \epsilon = R^*$

(8) $(PQ)^* P = P(QP)^*$

(9) $(P+Q)^* = (P^* + Q^*)^*$

$= (P^* Q^*)^*$

$= (P^* + Q)^*$

$= (P + Q^*)^*$

$= P^*(QP^*)^* = Q^*(PQ^*)^*$

(10) $(P+Q)R = PR + QR$

(11) $R(P+Q) = R \cdot P + R \cdot Q$

(10) write a RE for all strings in which 3rd symbol from RHS is 'a' over $\{a, b\}$.

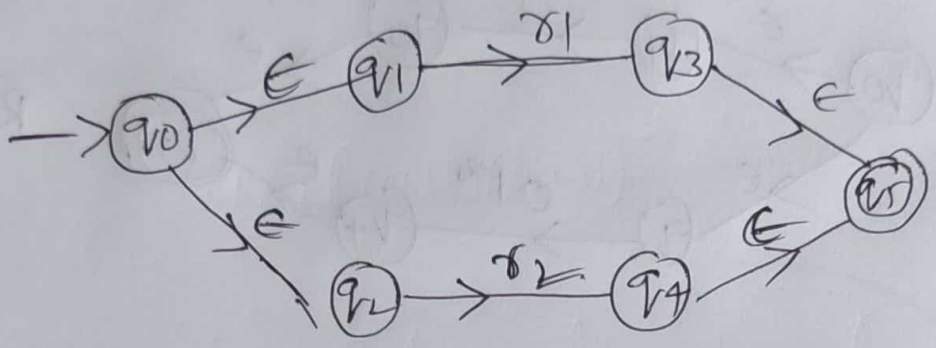
Sol:- Any(a+b) a (a+b) (a+b)

(one symb)

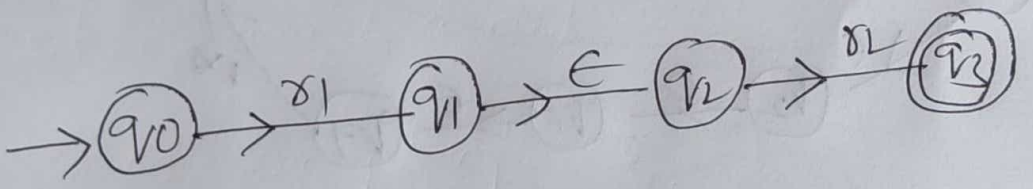
$RE = (a+b)^* a (a+b)(a+b)$

# Convert Regular Expression to ∈-NFA

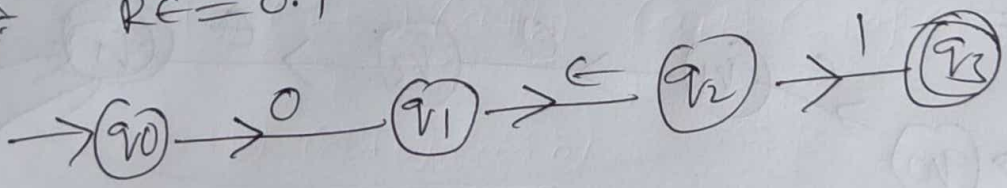
**Case 1:** $\gamma = \gamma_1 + \gamma_2$



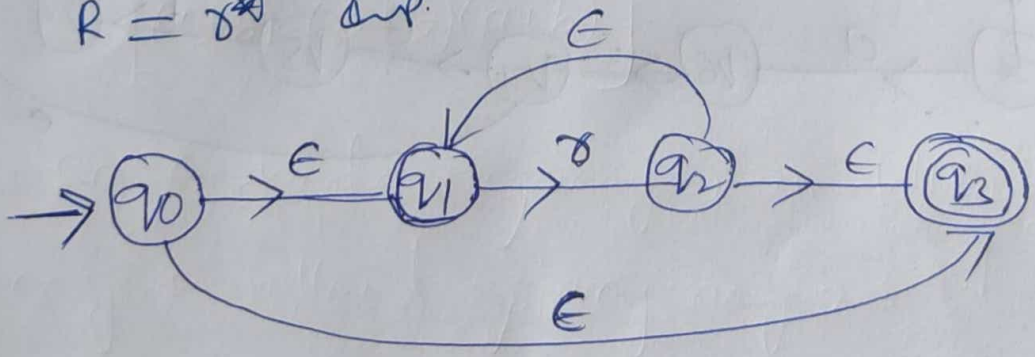

**Case 2:** $\gamma = \gamma_1 \cdot \gamma_2$



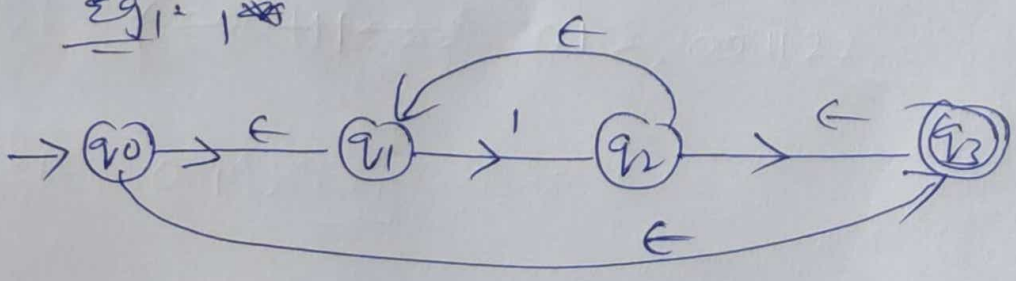

**Eg:** $RE = 0.1$




**Case 3:** $R = \gamma^*$ Kleene



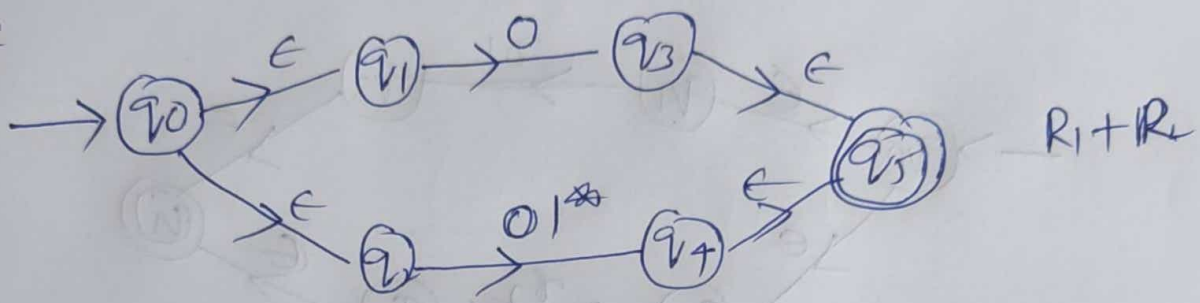

**Eg:** $1^*$



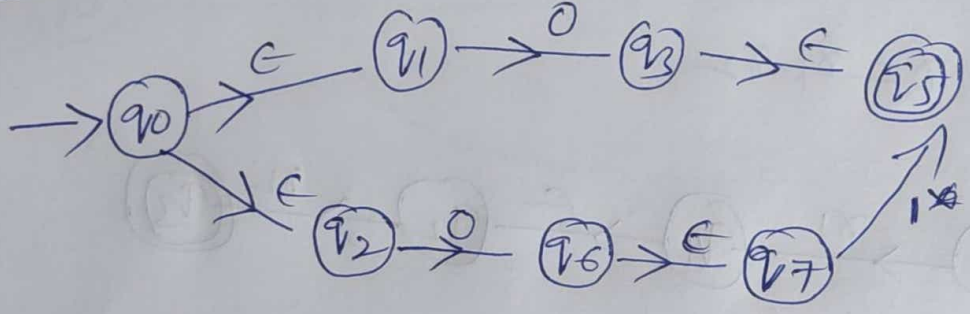
Eg1:- $R\epsilon = 0 + 01^*$ Convert into $\epsilon$-NFA

Sol- $R\epsilon = \underset{R_1}{0} + \underset{R_2}{01^*}$

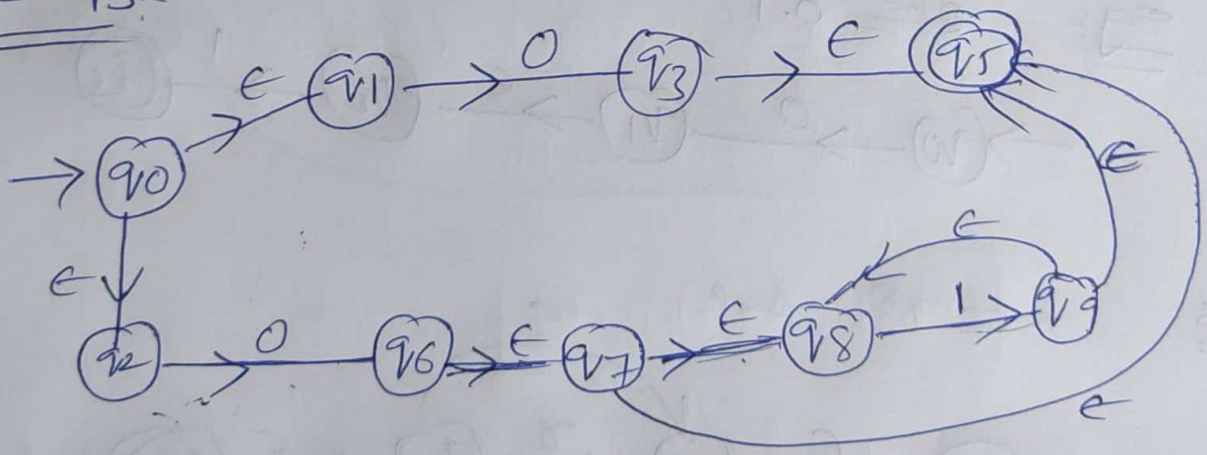
Step1:-




Step2:-

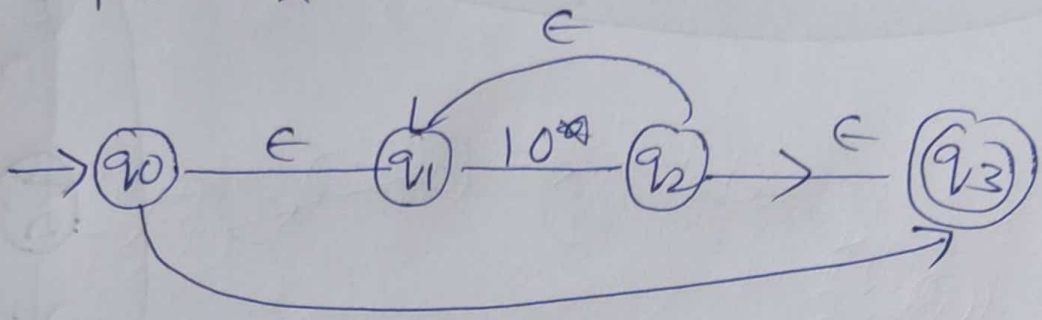


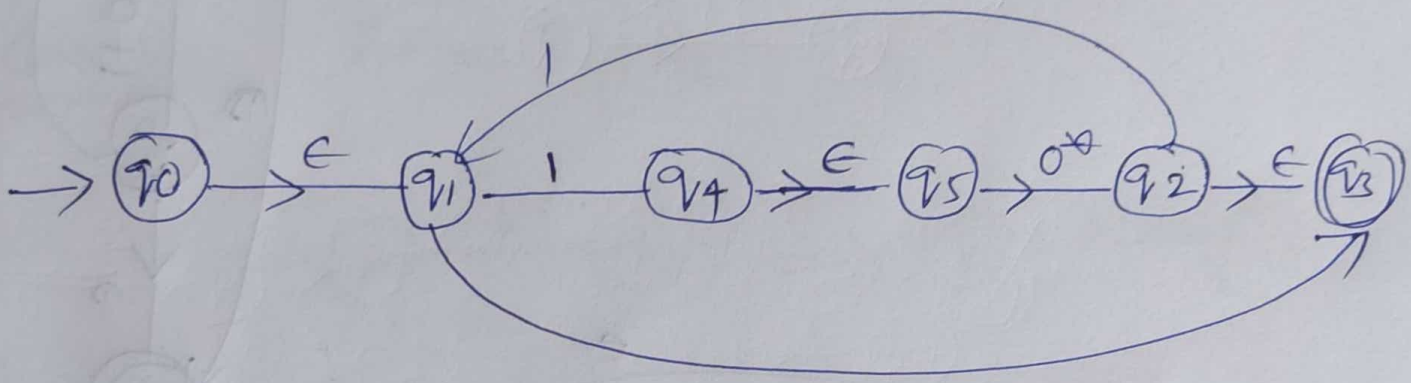

Step3:-

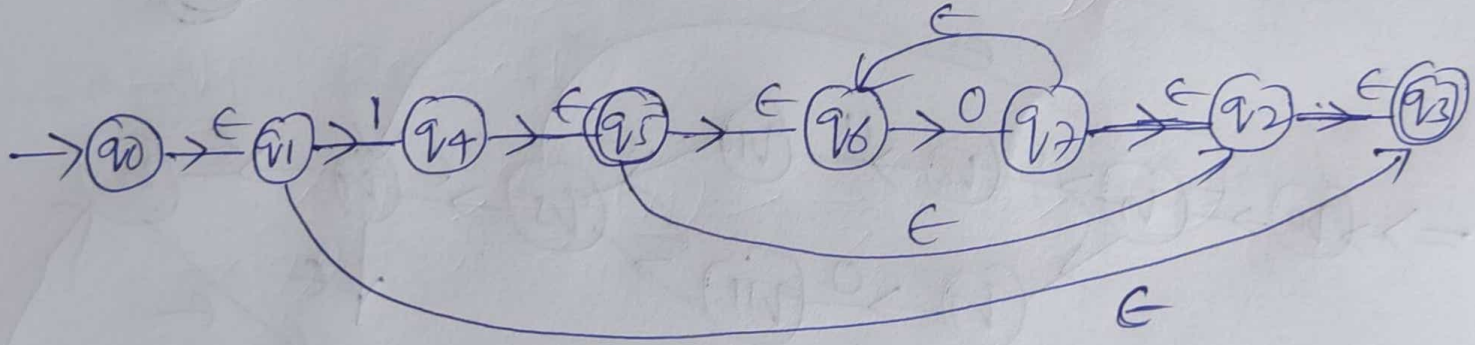


Eg2: $R\epsilon = (10^*)^*$

Sol:- $R\epsilon = R^*$

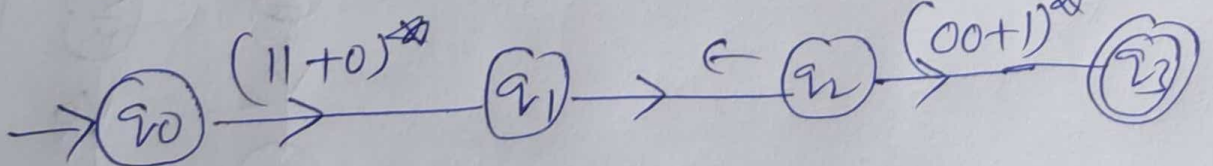


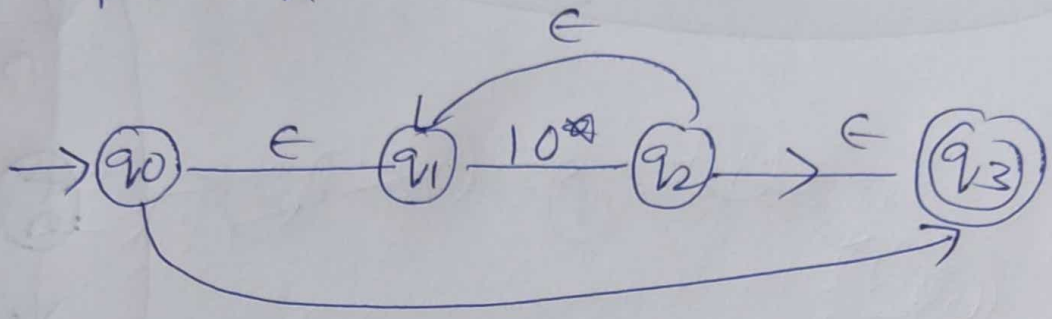

Eg3: $R\epsilon = (11+0)^*(00+1)^*$

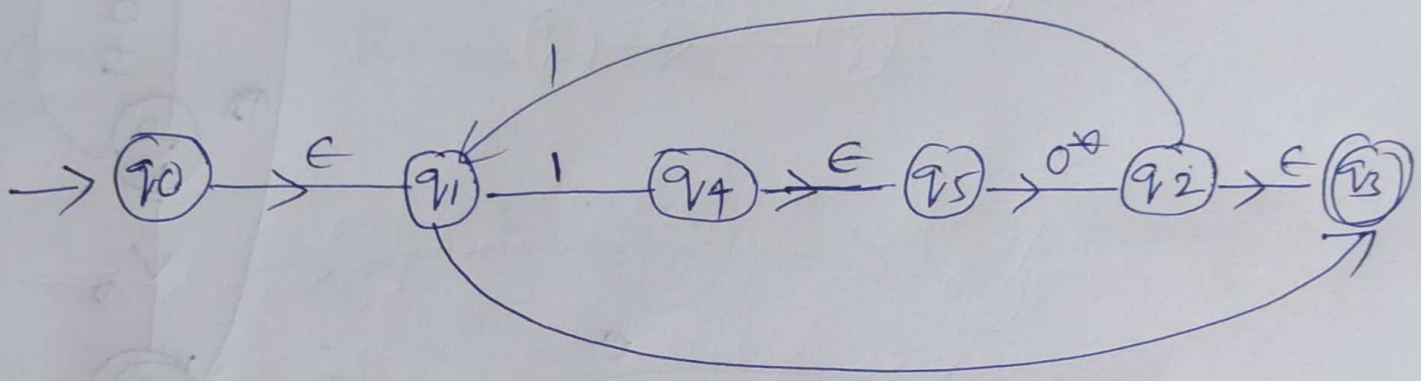
Sol:- $R = R_1 \cdot R_2$

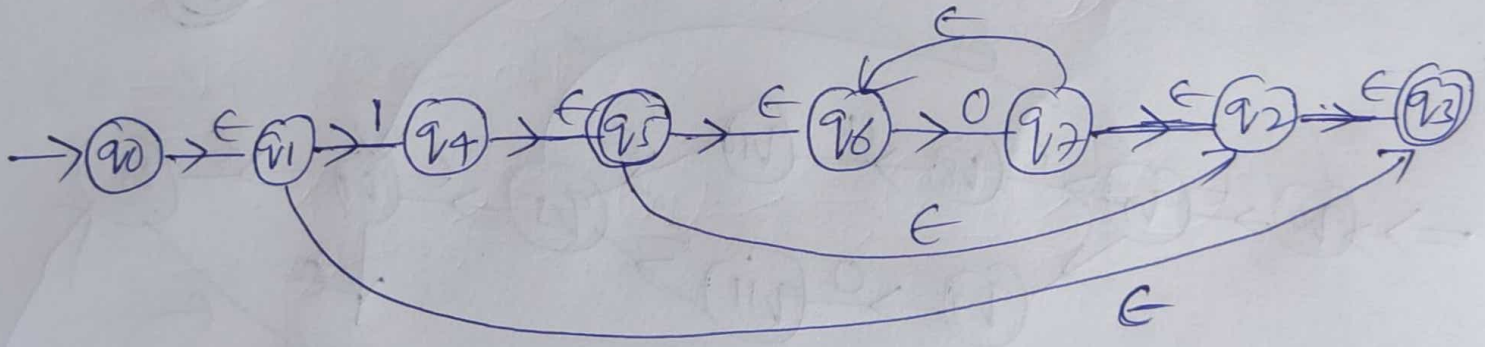


Eg2: $RE = (10^*)^*$

Sol:- $RE = R^*$

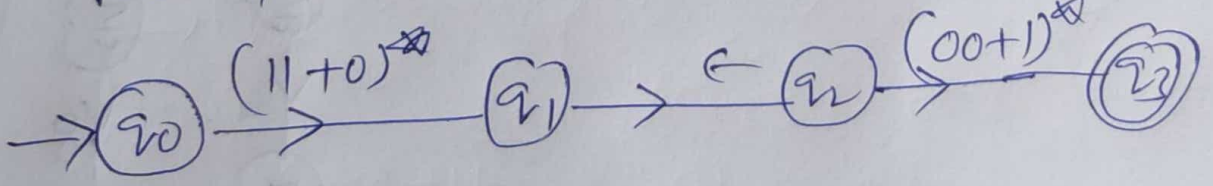


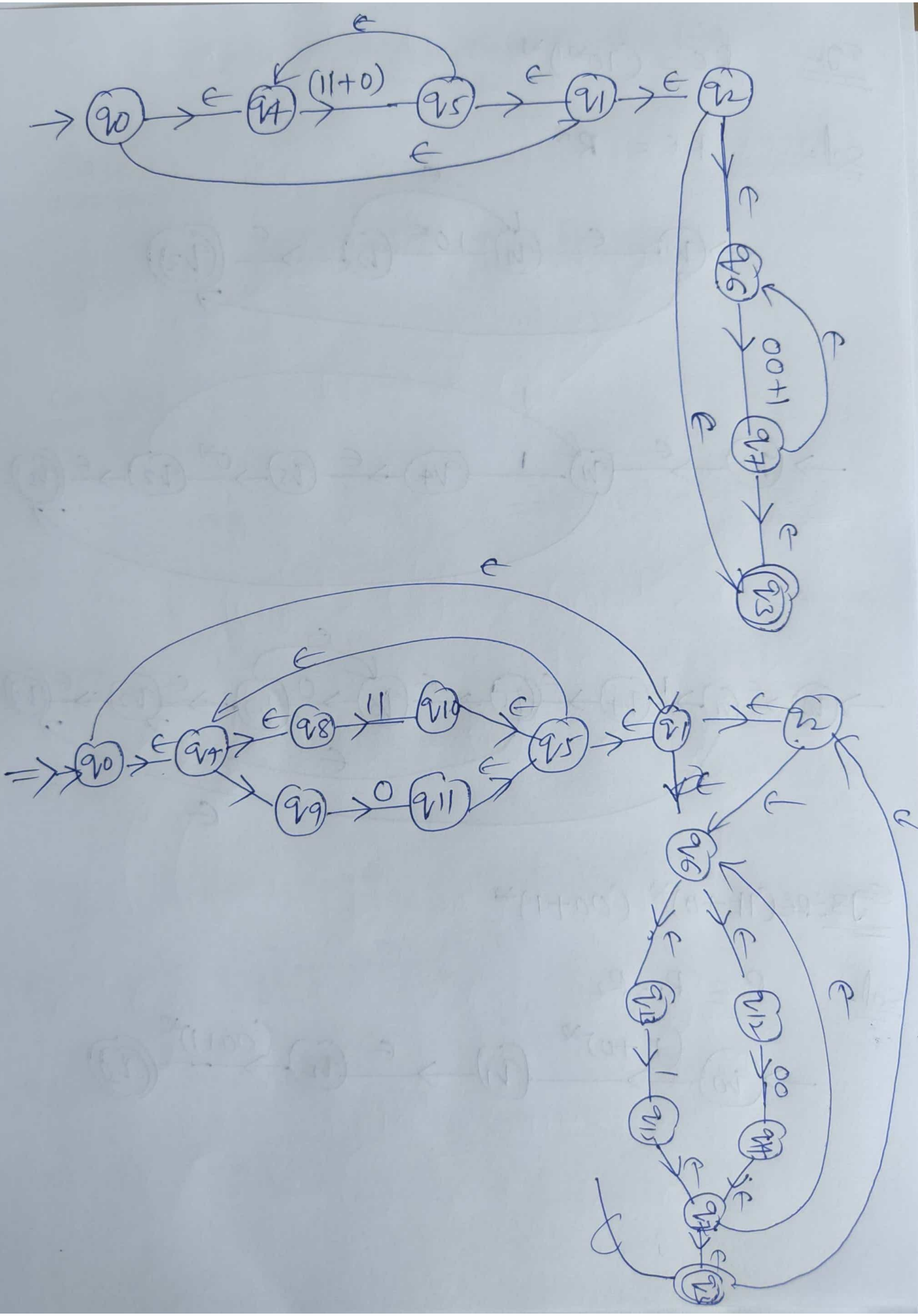




Eg3: $RE = (11+0)^*(00+1)^*$

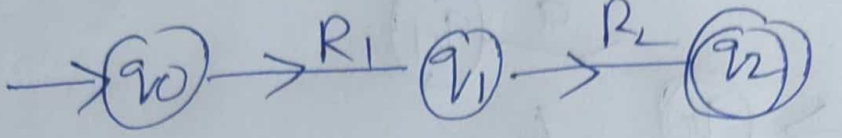
Sol:- $R = R_1 \cdot R_2$



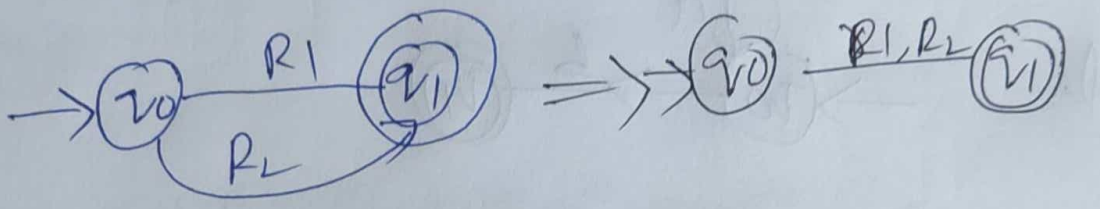

q0
q4
(11+0)
q5
q1
q2
q6
00+1
q7
q3
q0
q7
q8
11
q10
q5
q1
q2
q9
0
q11
q6
q12
1
q13
00
q14
q7
q4

# Convert Regular expression to FA

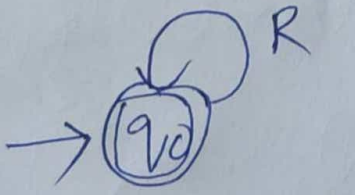
**Case 1 :** $RE = R_1 \cdot R_2$



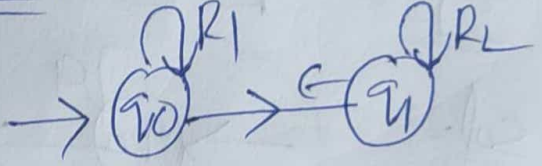

**Case 2 :** $RE = R_1 + R_2$



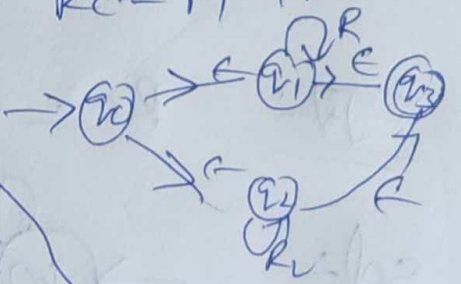

**Case 3 :** $RE = R^*$




**Case 4 :** $RE = R_1^* R_2^*$

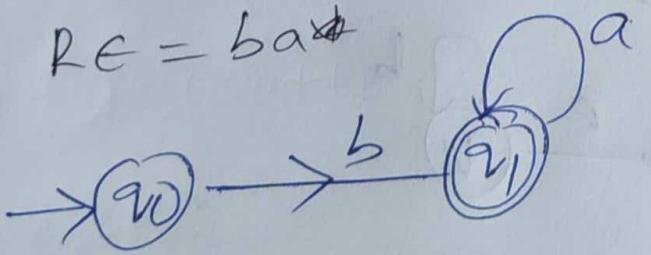



**Case 5 :** $RE = R_1^* + R_2^*$



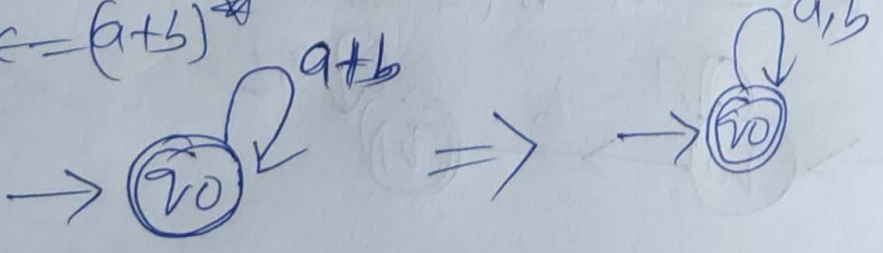

**Eg 1 :** $RE = ba^*$

**Sol :**



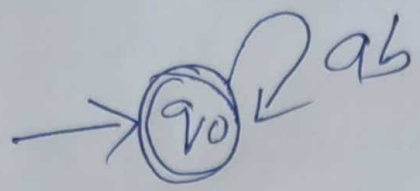

**Eg 2 :** $RE = (a+b)^*$

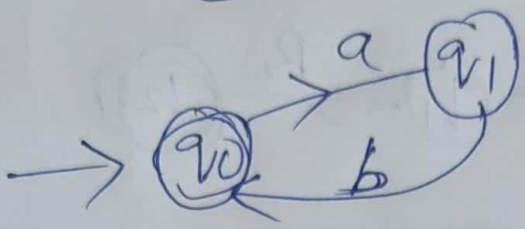
**Sol :**



Eg4:- $R.E = (ab)^*$

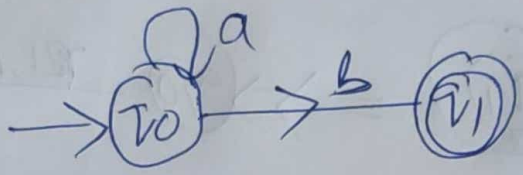
Sol:-



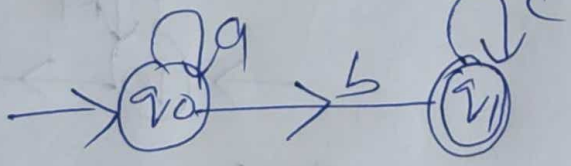

Eg5:- $R.E = a^* b$

Sol:-



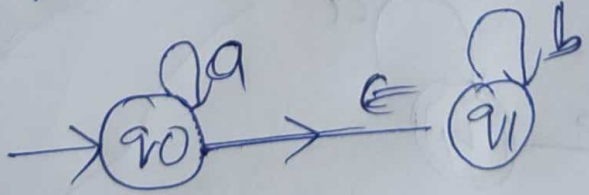

Eg6:- $R.E = a^* b c^*$

Sol:-



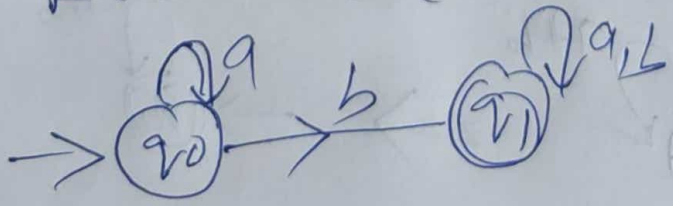

Eg7:- $R.E = a^* b^*$

Sol:-

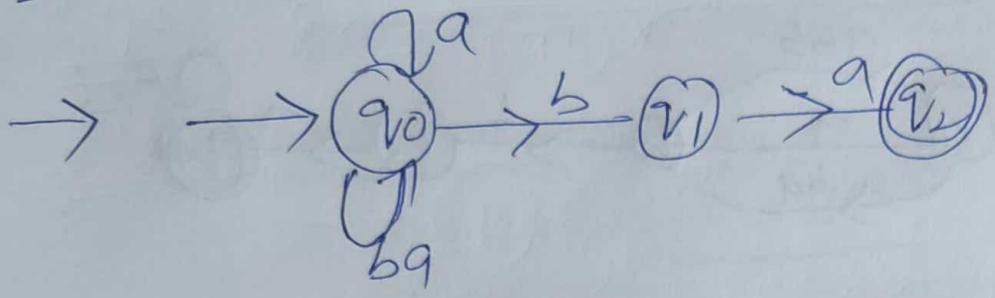


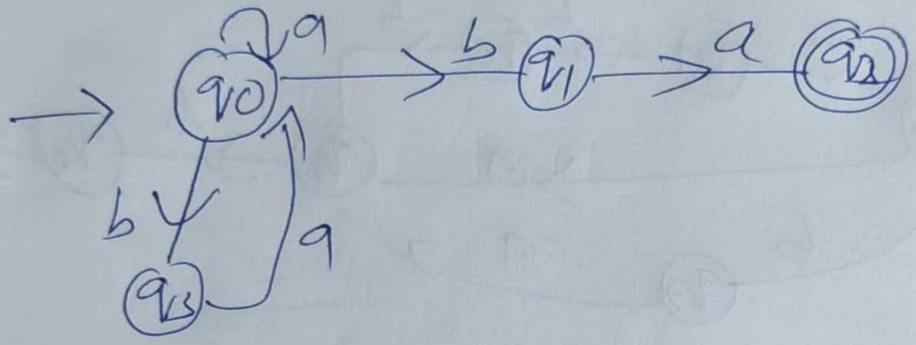

Eg8:- $R.E = a^* b (a+b)^*$

Sol-



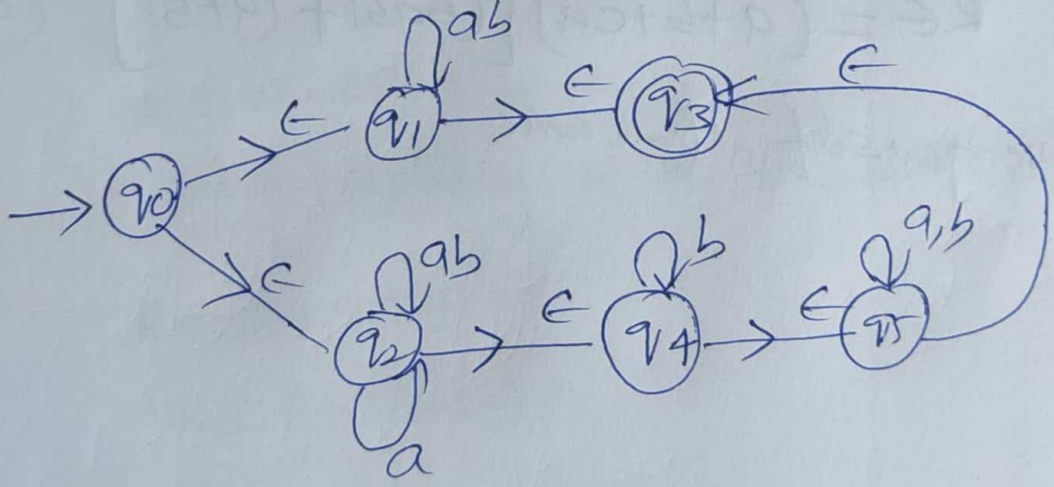
Eg9: $RE = (a+ba)^* ba$

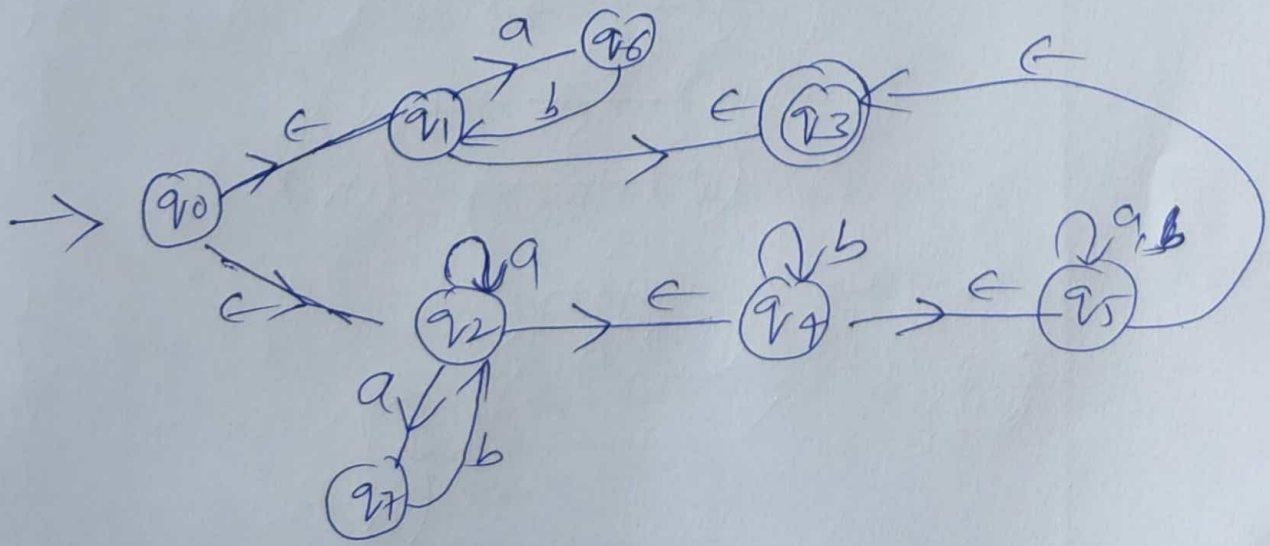



Eg10: $RE = (ab)^* + (a+ab)^* b^* (a+b)^*$

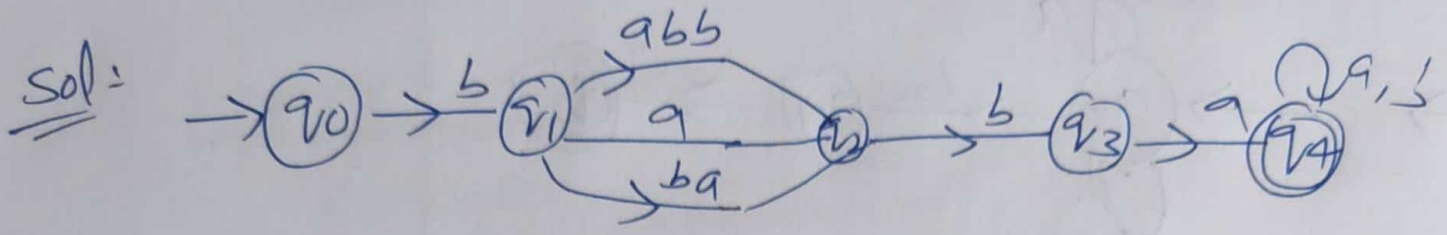
Sol:

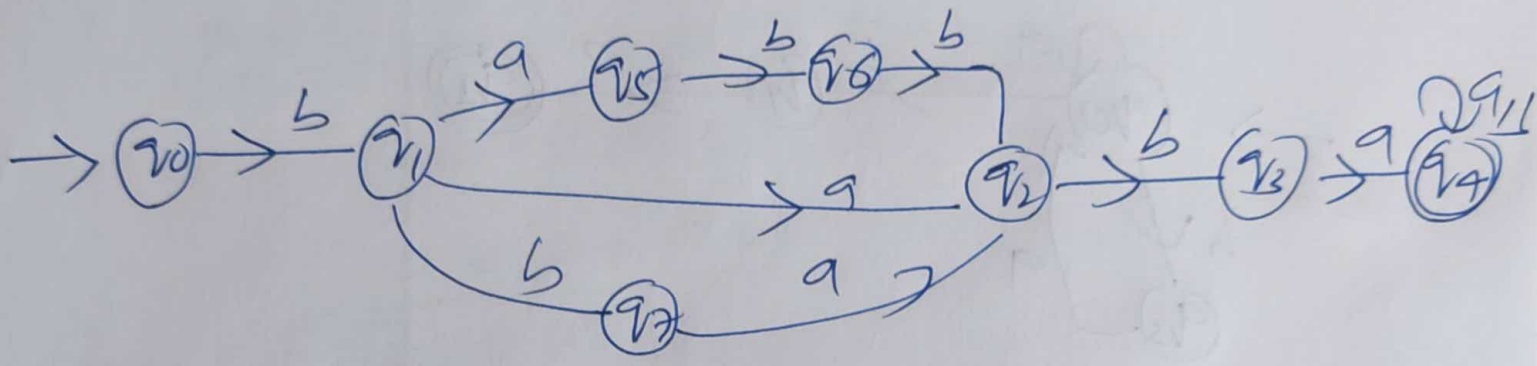


Eg 11:- $RE = b(a+ba+abb)ba(a+b)^*$

Sol:-




Eg 12:- $RE = (a+b+ca)\left[(bab) + (a+b)\right]^* (ab)^*$

→ Solve yourself, Home work

# Finite automata to RE

## Arden's Theorem:-

$R = Q + RP \Rightarrow \boxed{R = QP^*}$

**Proof 1:-**

$R = QP^*$ put in R

$R = Q + (QP^*)P$

$= Q(\epsilon + P^*P)$

$\boxed{R = QP^*}$

**Proof 2:-**

$R = Q + RP$ ——— (1)

Now replacing R by $R = Q + RP$

~~$R = Q + QP^*P$~~

$R = Q + (Q + RP)P$

$= Q + QP + RP^2$

$= Q + QP + [Q + RP]P^2$

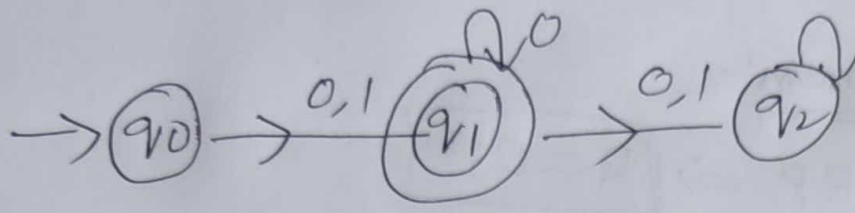
$= Q + QP + QP^2 + RP^3 \ldots$

$= Q + QP + QP^2 + QP^3 + QP^4 + RP^5 \ldots$

$R = Q[\epsilon + P + P^2 + P^3 + \ldots]$

$\boxed{R = QP^*}$

Eg1:- convert FA to RE




Sol:- First we have to write equations

$q_0 = \epsilon$ ——— (1)

$q_1 = q_0 0 + q_0 1 + q_1 0$ ——— (2)

$q_2 = q_1 0 + q_1 1 + q_2 1$ ——— (3)

From eqn (2)

$q_1 = q_0(0+1) + q_1 0$

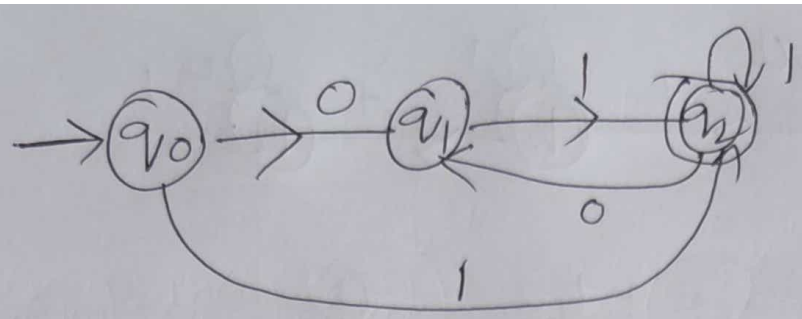
~~$R = Q + PR$~~ $\boxed{R = Q + RP}$ $\Rightarrow$ $\boxed{R = QP^*}$

$q_1 = q_0(0+1)0^*$

$q_1 = \epsilon(0+1)0^*$

$\boxed{RE = (0+1)0^*}$

Eg2:




Sol:

$q_0 = \epsilon$ ——— (1)

$q_1 = q_0 0 + q_2 0$ ——— (2)

$q_2 = q_1 1 + q_2 1 + q_0 1$ ——— (3)

$q_1 = \epsilon . 0 + q_2 0$

$\boxed{q_1 = 0 + q_2 0}$

$q_2 = q_0 1 + q_1 1 + q_2 1$

$= \epsilon . 1 + (q_2 0 + 0) 1 + q_2 1$

$= 1 + q_2 01 + 01 + q_2 1$

$q_2 = (1 + 01) + q_2 (01 + 1)$

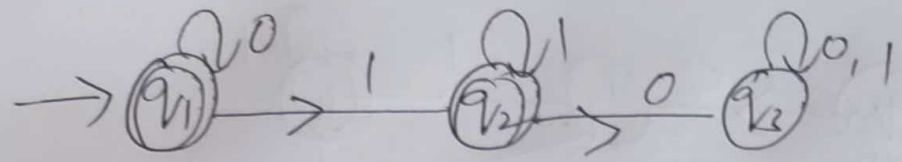
$R = Q + RP \Rightarrow R = QP^*$

$q_2 = (1 + 01)(01 + 1)^*$

$\boxed{R\epsilon = (1 + 01)^+}$

$\boxed{R\epsilon = R \cdot R^* = R^+}$

Eg3:-




Sol:-

$$q_1 = \epsilon + q_1 0 \quad \text{—(1)}$$

$$q_2 = q_1 1 + q_2 1 \quad \text{—(2)}$$

$$q_3 = q_2 0 + q_3 0 + q_3 1 \quad \text{—(3)}$$

Find $RE = q_1 + q_2$

$$q_1 = \epsilon + q_1 0$$

$$R = Q + RP \Rightarrow R = QP^*$$

$$q_1 = \epsilon 0^* = 0^*$$

$$q_2 = q_1 1 + q_2 1$$

$$q_2 = 0^* 1 + q_2 1$$

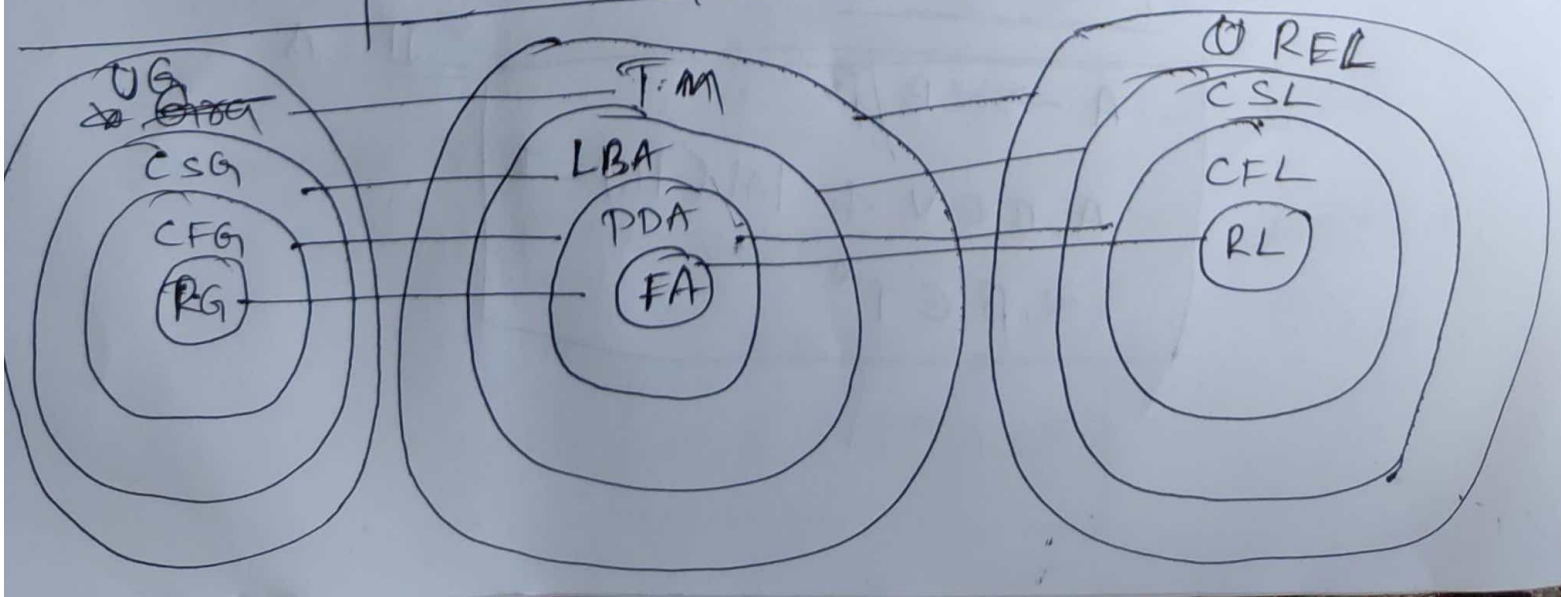
$$q_2 = 0^* 1 1^*$$

$$RE = 0^* + 0^* 1 1^*$$

# Grammars

→ The four types of Grammers according to Noam chomsky they are:

| Grammer type | Grammar accepted | Language accepted | Automatar (or) machine |
|---|---|---|---|
| ① Type-3 | ① Regular Grammer | ① Regular Language | Finite state automatar (or) NFA (or) DFA |
| Type-2 | Content Free Grammer | Content Free Language | Pushdown Automata |
| Type-1 | Content Sensitive Grammer | Content Sensitive Language | Linear Bounded automata |
| Type-0 | Unrestricted Grammar | Recursively Enumerable Language | Turing machine |



## Regular Grammar:

A Grammar "G" can be formally described using 4 tuples as $G=(V,T,S,P)$ where,

V = Set of Variables (or) Non-Terminals

T = Set of Terminals (or) Alphabet ($\Sigma$)

S = Start symbol

P = Production rules for Terminals and Non-terminal.

These are divided into 2 categories

1. Right Linear Grammar
2. Left Linear Grammar

(1) **Right Linear Grammar:** If the non-terminal symbol appears at right most symbol in each production then it is called Right Linear Grammar.

Production rules.

$A \to \alpha B / \beta$

$A, B \in V$ & $|A| \& |B| = 1$

$\alpha, \beta \in T^*$

Eg1: $\Lambda$

Eg1:- $A \to aB$

$A \to a$

$B \to \epsilon$

We know that $G_1 = (V, T, P, S)$

$V = \{A, B\}$ $T = \{a, \epsilon\}$

$P = \{A \to aB, A \to a, B \to \epsilon\}$

$S = \{A\}$

Yes It is Right Linear Grammar since in $A \to aB$ "B" is Right most Symbol.

$|A| \& |B| = 1$

$A \to a$ & $B \to \epsilon$ $A \to B$

Eg2:- $A \to aBb$

$B \to c/\epsilon$ is it RLG (or) not?

Sol:- It is not RLG b/c B is there in middle

② Left Linear Grammar:- If the non-terminal symbol appear as Left most symbol in each production of a regular grammar then it is called LLG.

Production Rule:-

$A \to B\alpha / B$

$A, B \in V$

$\alpha, \beta \in T^* (a, \Sigma^*$

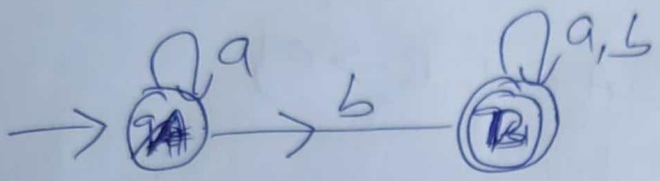
Eg1:- $A \to Ba$

$B \to b$

**Eg1:-** Construct the Right Linear Grammar for the regular Expression $a^* b(a+b)^*$

**Sol:-** $RE = a^* b(a+b)^*$

First construct the FA




$G = (V, T, P, S)$

$V = \{A, B\}$ States

$T = \{a, b\}$ (or) $\Sigma$

$S = \{A\}$

$P = ?$

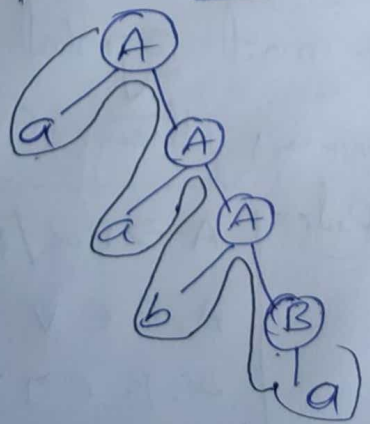
$A \rightarrow aA / bB / b$

$B \rightarrow aB / bB / a / b$

$w = aaba$

Step1 Starting symbol

Derivation Tree




$= aaba$ Yes

It is Generated

# Content Free Grammar (or) Type-2 (5)

→ CFG is defined by 4-tuples as $G = \{V, T, S, P\}$

$V$ = Set of Variables (or) non-Terminal

$T$ = Set of Terminal (or) Alphabet ($\Sigma$)

$S$ = Start Symbol

$P$ = Production Rule.

Content Free Grammar has production Rule of the form

$$A \rightarrow \beta$$

$A \in V, \quad |A| = 1$

$\beta \in (T \cup V)^*$

Eg1:

$S \rightarrow aAb$

$A \rightarrow bA/a$ → It is CFG.

Note:

→ CFG it generate content free language which is accepted by push-down automata

check the given Grammar is CFG (or) not.

CFG → CFL → PDA

Eg2: $S \rightarrow asb/ab$

Eg3: $AB \rightarrow aABb/\epsilon$

Eg4

$A \rightarrow aABb/c$

$B \rightarrow bBb/\epsilon$

## Derivation trees:

A derivation Tree (or) parse Tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a Context Free Grammar.

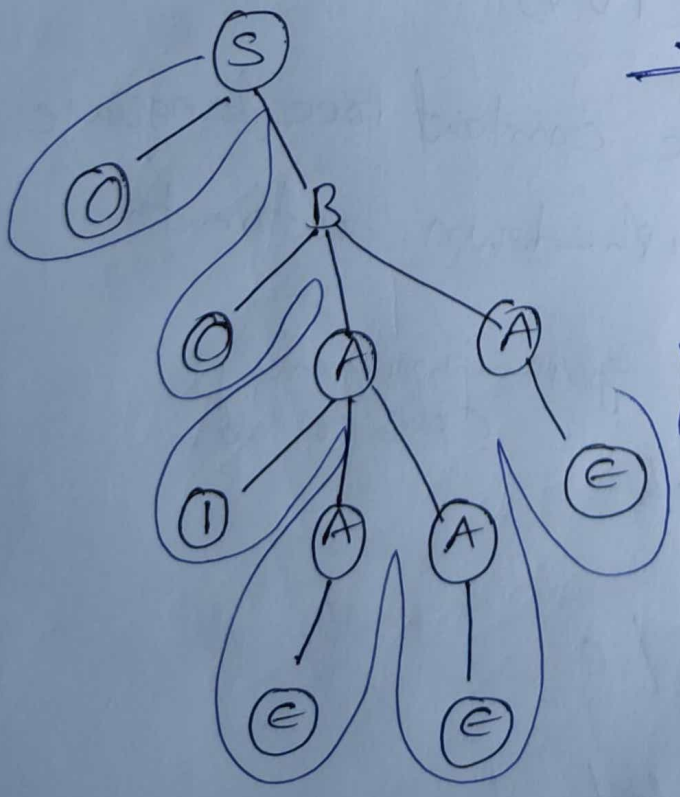
**Eg:** For the Grammar $G = \{V, T, P, S\}$ where

$S \to 0B$, $A \to 1AA/\epsilon$, $B \to 0AA$

**Root vertex:** Must be labelled by the start symbol.

**Vertex:** Labelled by Non-Terminal symbols

**Leaves:** Labelled by terminal symbol (or) $\epsilon$



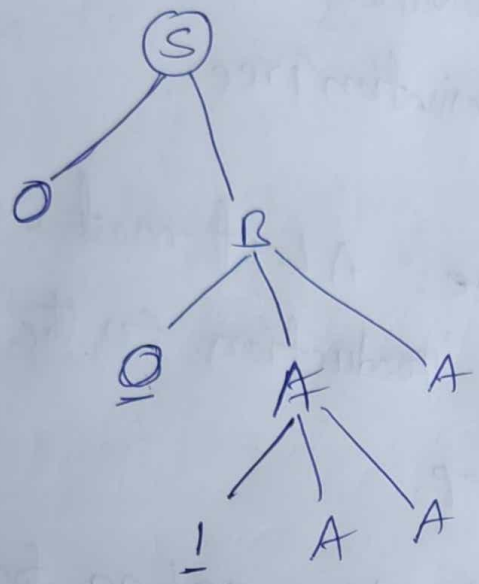

→ This parse is driving the string

0 0 1 ε ε ε

⇒ 0 0 1

→ For the above Grammar delive ~~the~~ can you derive the

W = 00011



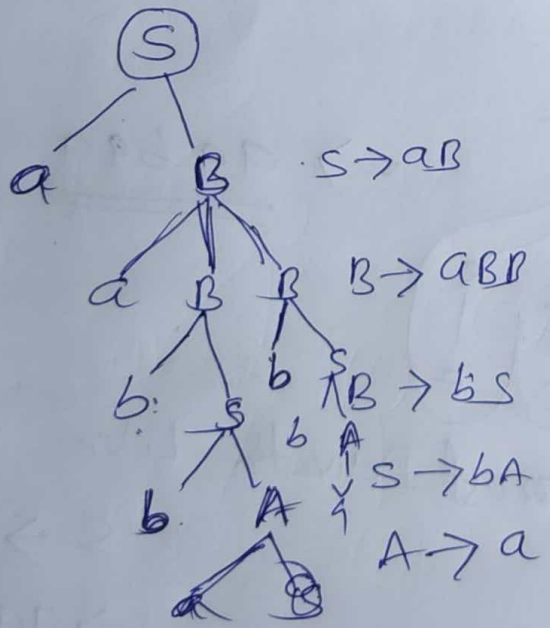

→ In this Grammar after '00' compulsory coming '1' so we can't delive the 00011. This string is not belongs to the Gramma.

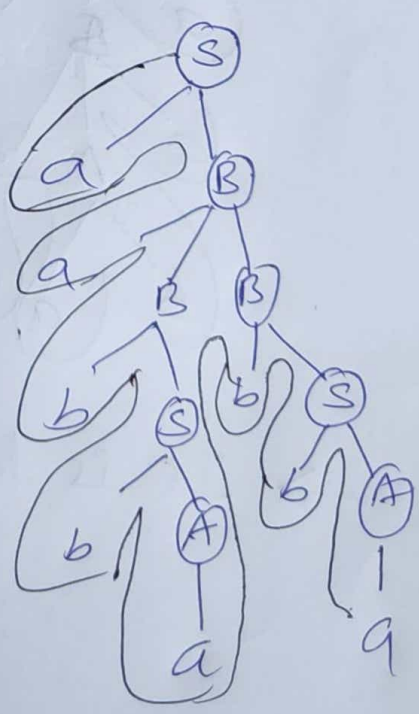
Eg2: construct a delivation tree (or) parsetree for the string aabbabba for C.F.G S → aB/bA

A → a/aS/bAA

B → b/bS/aBB

Sol:



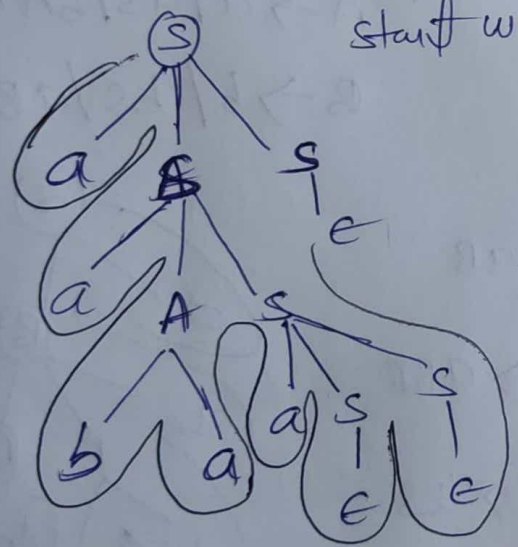

aabbabba

→ String is delived.

Def Derivation Tree can be obtained by (8)
2 ways
(1) Left most Derivation Tree
(2) Right most Derivation Tree.

(1) Left most Derivation Tree: A Left most Derivation Tree is obtained by applying production to the leftmost Variable in each step.

Eg1:- For generating the string aabaa from the Grammar S → aAS / ass / ϵ,
A → SbA / ba
w = aabaa
start with starting "symbol"




=> aabaa

→ It is left most Derivation Like S → aSS
S → aaASS
S → aabaSS
S → aabaaSSS
S → aabaaϵϵϵ
S → aabaa

## (2) Rightmost Derivation Tree:-

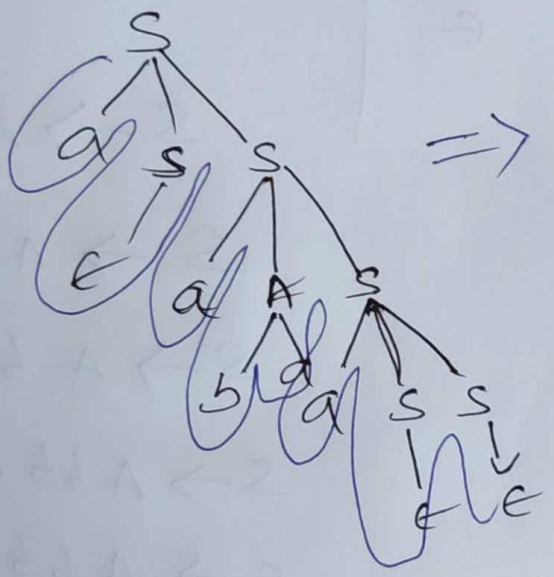
A Rightmost Derivation Tree is obtained by applying production to the rightmost variable in each step.

**Eg:-** For generating the string aabaa from the Grammar

$S \rightarrow aAS \mid ass \mid \epsilon$

$A \rightarrow SbA \mid ba$

~~ww~~ = aabaa




$\Rightarrow$ aabaa

$S \rightarrow asS$

$S \rightarrow asaAS$

$S \rightarrow asaAass$

$S \rightarrow asaAas\epsilon$

$S \rightarrow asaAa\epsilon$

$S \rightarrow asaAa$

$S \rightarrow asabaa$

$S \rightarrow a\epsilon abaa \quad S$

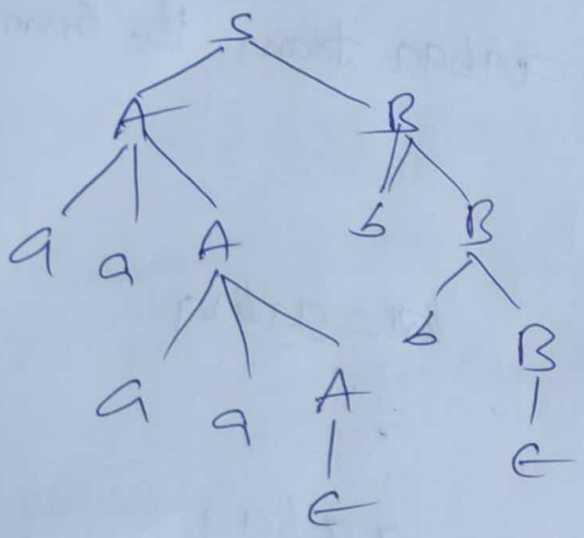
$S \rightarrow aabaa$

Eg3. Derive LMD & RMD for $S \to AB$, $A \to aaA/\epsilon$, $B \to bB/\epsilon$

$W = aaaabb$

Sol:

LMD




$S \to \underline{A}B$

$S \to aa\underline{A}B$

$S \to aaaa\underline{A}B$

$S \to aaaa\epsilon B$

$S \to aaaaB$

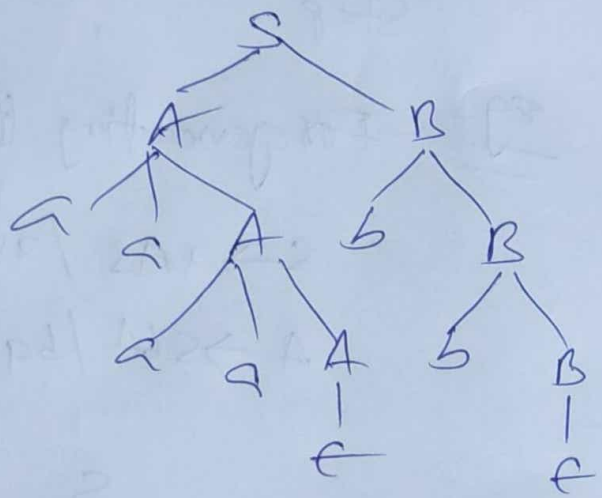
$S \to aaaabB$

$S \to aaaabbB$

$S \to aaaabb\epsilon$

$S \to aaaabb$

RMD




$S \to A\underline{B}$

$S \to Abb\underline{B}$

$S \to Abbb\epsilon$

$S \to Abbb$

$S \to aa\underline{A}bbb$

$S \to aaaaAbbb$

$S \to aaaa\epsilon bbb$

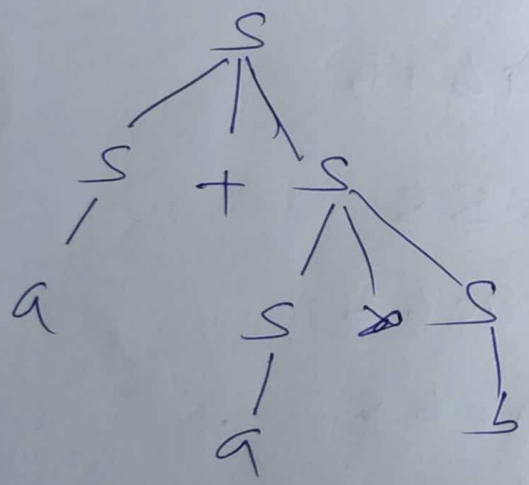
$S \to aaaabbb$

# Ambiguous Grammar

→ A Grammar is said to be Ambiguous if there exists two (or) more derivation tree for a string w that means two (or) more leftmost derivation trees.

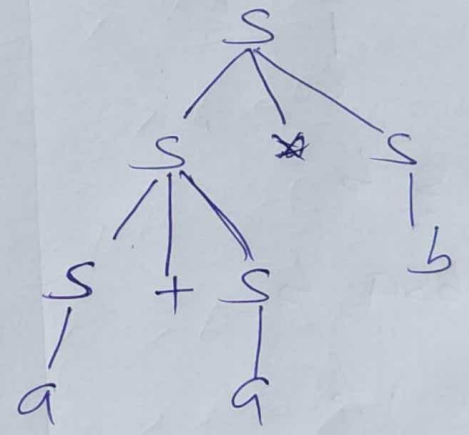
Eg: $G = \{\{S\}, \{a, b, +, *\}, P, S\}$ where P consists of $S \to S+S \mid S*S \mid a \mid b$ the string $a+a*b$ can be generated as:

LMD1




$a+a*b$ by one LMD1

LMD2




$a+a*b$ by one LMD2

→ For this string we have two parse tree so it is ambiguous grammar.

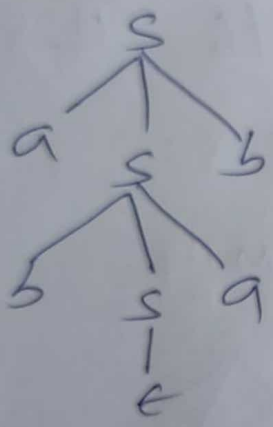
Eg2: $G = \{\{S\}, \{a,b\}, \{P\}, \{S\}\}$ P consists

V T S

$S \to asb \mid bsa \mid ss \mid \epsilon$ String $w = abab$

Sol. $w = abab$

LMD1




$S \to asb$

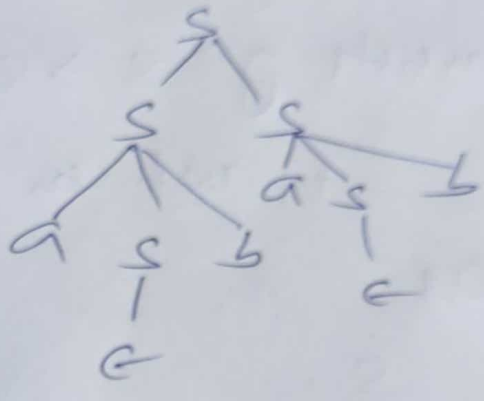
$S \to abSab$

$S \to abεab$

$S \to abab$

$w = abab$

LMD2




$\Rightarrow abab$

$S \to SS$

$\to asbS$

$\to aεbS$

$\to abS$

$\to abasb$

$\to abaεb$

$\to$ abab

→ It has two LMDs so it is ambiguous grammar

# Simplification of context Free Grammar

## (or) Reduce of CFG.

→ In CFG, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some NULL productions and UNIT productions. Elimination of these productions and symbol is called simplification of CFG.

Simplification consists of the following steps.

(1) Removal of Null production (or) $\in$ - production

(2) Removal of Unit productions.

(3) Reduction of CFG (or) elimination of useless production.

### (1) Remove of Null productions:-

Step 1:- To Remove $A \to \in$, look for all production whose right side contains A

Step 2:- Replace each occurence of "A" in each of these production with $\in$

Step 3:- Add the resultant production to the Grammar

Eg1:- Remove Null production from the following Grammar.

$S \to ABAC$, $A \to aA/\epsilon$, $B \to bB/\epsilon$, $C \to c$

Sol:- $A \to \epsilon$, $B \to \epsilon$

(1) To eliminate $A \to \epsilon$

$S \to \underline{A}BAC$ If you ~~sub~~ replace with $A \to \epsilon$ we get diffut production

$S \to BAC/ABC/BC$ —— (1)

~~$A \to aA$~~ $A \to aA$

~~$S \to$~~ $A \to a$ —— (2)

~~New~~ New production are

$S \to BAC/ABC/BC/ABAC$

$A \to a/aA$

$B \to bB/\epsilon$, $C \to c$

(2) To eliminate $B \to \epsilon$

$S \to BAC$

$S \to AC$ —— (1)

$S \to ABC$

$S \to AC$ —— (2)

$S \to BC$

$S \to c$ —— (3)

$B \to bB$

$B \to b$

$S \to ABAC$

$S \to AAC$

After eliminating $B \to \epsilon$

$S \to BAC / ABC / BC / ABAC / AC / \emptyset / AAC / C$

$B \to bB / b$

$A \to a / aA$

Eg 2: $S \to AB,\ A \to aAA / \epsilon,\ B \to bBB / \epsilon$

Sol: Nullable variable $= \{A, B, S\}$

$S \to \epsilon.\epsilon,\ S \to \epsilon$

$S \to AB / B / A / \epsilon$

$A \to aAA / aA / a$

$B \to bBB / bB / b$

## Removal of unit productions

→ Any production Rule of the form $A \to B$ where $A, B \in$ Non-Terminals is called unit production

### procedure for Removal:

Step 1: To remove $A \to B$, add production $A \to x$ to the grammar rule whenever $B \to x$ occurs in the Grammar [$x \in$ Terminal, $x$ can be null)

Step 2: Delete $A \to B$ from the Grammar & Repeat

Eg1:- Remove unit production from the Grammar whose production rule is given by

P: $S \to XY$, $X \to a$, $Y \to Z/b$, $Z \to M$,
$M \to N$, $N \to a$

Sol:- unit production are

$Y \to Z$, $Z \to M$, $M \to N$.

Take $Y \to Z \Rightarrow Y \to M \Rightarrow Y \to N$

$\Rightarrow Y \to a$ so we can write $\boxed{Y \to a}$

$Z \to M \Rightarrow Z \to N \Rightarrow Z \to a$

$\boxed{Z \to a}$

$M \to N \Rightarrow \boxed{M \to a}$

so finally our production are

P: $S \to XY$, $X \to a$, $Y \to a/b$, $Z \to a$, $M \to a$

Eg2:- $S \to Aa/B$
$B \to A/bb$
$A \to a/bc/B$

Sol:- unit production are

$A \to B,\ B \to A,\ S \to B$

$A \to B \Rightarrow \boxed{A \to bb}$

$B \to A \Rightarrow \boxed{B \to a \mid bc}$

$S \to B \Rightarrow \boxed{S \to bb \mid a \mid bc}$

Finally production are

$S \to Aa \mid bb \mid a \mid bc$

$B \to a \mid bc \mid bb$

$A \to a \mid bc \mid bb$

## ③ Elimination of the useless productions.

→ A symbol can be useless if it does not appear on the right-hand side of the production rule and does not take part in the derivation of any string. That symbol is known as a useless symbol. Similarly a variable can be useless if it does not take part in the derivation of any string. That variable is known as a useless variable.

**Eg:** $S \to AC/B$ $A \to a$, $C \to d/BC$, $E \to aA/e$

Eliminate useless productions.

**Sol:**

**Step1** - First write the terminal as one set

$T = \{a, d, e\}$

**Step2** - write the non-terminal, which production are derivable the terminal

$= \{a, d, e, A, C, E\}$ so A, C, E are use full symbol.

**Step3** - write the Combinations useful production Combination of above sol-

$= \{a, d, e, A, C, E, S,\}$ → Finally A, C, E, S Variables are useful all other not useful that is 'B' so with out Right side where if 'B' is there eliminate that productions.

Finally production are

P: $S \to Ac$, $A \to a$, $c \to d$, $E \to aA/e$

Note: $S \to B$, $C \to BC$ are useless productions

Eg2: $S \to AB/Ac$

$A \to aAb/bAa/a$

$B \to bbA/aaB/AB$

$C \to abCA/aDb$

$D \to bD/ac$ eliminate useless productions

Sol: Step1: Terminals $= \{a, b\}$

Step2: useful symbols $= \{a, b, A\}$

Step3: Combination terminal & variable can generate symbol

$= \{a, b, A, B, S\}$ so A, B, S are useful

other variables C, D are not useful so at Right side C, D is there eliminate that production

So Finally productions are:

$S \to AB$

$A \to aAb/bAa/a$

$B \to bbA/aaB/AB$

$S \to Ac$

$C \to abCA/aDb$

$D \to bD/ac$ are useless productions.

# Chomsky Normal Form

-> In chomsky Normal Form (CNF) we have a restrication on length of RHS, which is elements in RHS should either be two variable (or) Terminal.

-> A CFG is in chomsky Normal Form if the production are in the following forms

$A \to BC$ where A, B, C are non-terminal
$A \to a$ a is a terminal.

## Steps to Convert a given CFG to chomskey Normal Form.

**Step1**: If the start symbol "S" occcurs on some right side, create a new start symbol S' and a new production $S' \to S$

**Step2**: Remove Null production.

**Step3**: Remove unit production.

**Step4**: Replace each production $A \to B_1 \cdots B_n$ where $n > 2$ with $A \to B_1 C$, where $C \to B_2 \cdots B_n$ Repeat this steps.

**Step5**: If the right side of any production is in the form $A \to aB$ where "a" is terminal and replaced by $A \to XB$ and $X \to a$ Repeat this step

Eg1:- Convert the following CFG to CNF

P: $S \to ASA/aB$, $A \to B/S$, $B \to b/\epsilon$

Sol:- "S" appears in RHS, we add a new state $S'$ and $S' \to S$

P: $S' \to S$, $S \to ASA/aB$, $A \to B/S$, $B \to b/\epsilon$

Step2- Remove the null production.

$B \to \epsilon$, $A \to \epsilon$

After Removing $B \to \epsilon$: P: $S' \to S$, $S \to ASA/aB/a$,

$A \to B/S/\epsilon$, $B \to b$

After Removing $A \to \epsilon$:

P: $S' \to S$, $S \to ASA/aB/a/SA/AS/S$,

$A \to B/S$, $B \to b$

Step3- Remove the unit production: $S' \to S$, $S \to S$

$A \to B$ and $A \to S$

After Removing $S \to S$: P: $S' \to S$, $S \to ASA/aB/a/SA/AS$

$A \to B/S$, $B \to b$

After Removing $S^1 \to S$

P: $S^1 \to ASA/aB/a/AS/SA$

$S \to ASA/aB/a/AS/SA,$

$A \to B/S$, $B \to b$

After Removing $A \to B$

P: $S^1 \to ASA/aB/a/AS/SA$

$S \to ASA/aB/a/AS/SA$

$A \to b/S$, $B \to b$

After Removing $A \to S$

P: $S^1 \to ASA/aB/a/AS/SA$

$S \to ASA/aB/a/AS/SA$

$A \to b/ASA/aB/a/AS/SA$, $B \to b$

Step 4. NOW Find out the productions that has more than two variables in RHS

$S^1 \to ASA$, $S^1 \to ASA$, $A \to ASA$

After removing there we get P: $S^1 \to AX/aB/a/AS/SA$

$S \to AX/aB/a/AS/SA$

$A \to b/AX/aB/a/AS/SA$

$B \to b$

$X \to SA$

Step 5: Now change the production

$S' \to aB$, $S \to aB$ and $A \to aB$

Finally we get P:

$S' \to AX \mid YB \mid a \mid AS \mid SA$

$S \to AX \mid YB \mid a \mid AS \mid SA$

$A \to b \mid AX \mid YB \mid a \mid AS \mid SA$

$B \to b$,

$X \to SA$

$Y \to a$ Here all production are in CNF

Eg:- Convert the following CFG to CNF P:

$S \to ABA$, $A \to aA/\epsilon$, $B \to bB/\epsilon$

Sol:- Step 1:- "S" not appear Right hand side so not required new state but $S \to \epsilon$ so add new start

$S' \to S$

Step 2:- Remove null production that are

$A \to \epsilon$, $B \to \epsilon$

$S' \to S$

Remove $A \to \epsilon$:- P: $S \to ABA/BA/AB/B$,

$A \to aA/a$

$B \to bB/\epsilon$

Remove $B \to \epsilon$:- P: $S \to ABA/BA/AB/B/AA/A/\epsilon$

$A \to aA/a$, $B \to bB/b$

$S' \to S$

Remove $S \to \epsilon$:-

P: $S \to ABA/BA/AB/B/AA/A$

$A \to aA/a$, $B \to bB/b$

$S' \to ABA/BA/AB/B/AA/A$

Step 3:- Remove Unit product are $S \to A$, $S' \to S$,

$S \to B$, $S' \to A$

After $S \to A$, $S \to B$, $S' \to S$

P: $S \to ABA \mid BA \mid AB \mid B \mid AA \mid aA \mid a$

$A \to aA \mid a$, $B \to bB \mid b$

$S' \to ABA \mid BA \mid AB \mid B \mid AA \mid A$

After Removing $S' \to A$ & $S \to B$ & $S' \to B$

P: $S \to ABA \mid BA \mid AB \mid bB \mid b \mid AA \mid aA \mid a$

$A \to aA \mid a$, $B \to bB \mid b$

$S' \to ABA \mid BA \mid AB \mid bB \mid b \mid AA \mid aA \mid a$

→ In the above production $S \to ABA$, $S' \to ABA$ not following CNF so convert into CNF

$S \to XA \mid BA \mid AB \mid bB \mid b \mid AA \mid aA \mid a$

$A \to aA \mid a$, $B \to bB \mid b$

$S' \to XA \mid BA \mid AB \mid B \mid AA \mid aA \mid a \mid bB \mid b$

$X \to AB$

In the above production $S \to bB \mid aA$ & $B \to bB$, $S' \to aA$, $S' \to bB$ not following CNF

$K \to b$, $L \to a$

$S \to KB \mid LA$ $S' \to LA \mid KB$

$B \to KB$

So Finall production are

$S \to XA \mid BA \mid AB \mid KB \mid LA \mid AA \mid a$

$A \to LA \mid a \quad B \to KB \mid b$

$S' \to XA \mid BA \mid AB \mid B \mid AA \mid LA \mid a \mid KB \mid b$

$X \to AB$

$K \to b, \quad L \to a$

# Greibach Normal form

-> A CFG is in greibach Normal form if the productions are in the following forms:

$$A \rightarrow bC_1C_2 \cdots C_n$$

$$A \rightarrow b$$

where $A, C_1 \cdots C_n$ are Non-Terminals and $b$ is terminal

## Steps to Convert CFG to GNF-

**Step1:** Check whether the CFG is already in CNF and Convert it to CNF if it is not.

**Step2-** change the names of the Non-Terminal symbols into some $A_i$ in ascending order of $i$

**Step3:** If the production is of the form $A_i \rightarrow A_jX'$, then $i<j$ and should never be $i \geq j$.

**Step4:-** Remove left Recursion.

How to Remove Left Recursion

Eg: $A \to A\alpha \mid a \mid b$

$A' \to \alpha A' \mid \epsilon$

$A \to aA' \mid bA'$

→ Remove Null production that is $A' \to \epsilon$

⇓

$A' \to \alpha A' \mid \alpha$

$A' \to aA' \mid bA' \mid a \mid b$

Eg1: Convert following CFG to GNF

P: $S \to AA \mid 0$

$A \to SS \mid 1$

Sol: Step1: Already CFG has CNF

Step2: change the Names $S - A_1$, $A - A_2$

P: $A_1 \to A_2 A_2 \mid 0$ $\quad i < 2$

$A_2 \to A_1 A_1 \mid 1$ $\quad 2 > 1$ not allowed

Step3: $A_i \to A_j X$ $\quad i \le j$ allowed, $i \ge j$ not allowed

So Substitute $A_1$

$A_2 \to A_2 A_2 A_1 \mid 0A_1 \mid 1$

$A_2 \rightarrow \underline{A_2} \underline{A_2 A_1} / \underline{0A_1} / 1$

$\underline{A}$ $\alpha$

$2 \geq 2$ not allowed

Eliminate Left Recursion

$A \rightarrow A\alpha / a / b$

$A' \rightarrow \alpha A' / \epsilon$

$A' \rightarrow aA' / bA'$

$A' \rightarrow A_2 A_1 A' / \epsilon$

$A_2 \rightarrow 0A_1 A' / \cancel{bA} / 1A'$

Eliminate $A' \rightarrow \epsilon$

$A' \rightarrow A_2 A_1 A' / A_2 A_1$

$A_2 \rightarrow 0A_1 A' / 1A' / 0A_1 / 1$

P: $A_1 \rightarrow \underline{A_2} A_2 / 0$

$A' \rightarrow \underline{A_2} A_1 A' / \underline{A_2} A_1$

$A_2 \rightarrow 0A_1 A' / 1A' / 0A_1 / 1$ — GNF

Substitute "$A_2$" in $A'$ & $A_1$

$A_1 \rightarrow 0A_1A'A_2 / 1A'A_2 / 0A_1A_2 / 1A_2 / 0$

$A' \rightarrow 0A_1A'A_1A' / 1A'A_1A' / 0A_1A_1A' / 1A_1A' /$

$0A_1A'A_1 / 1A'A_1 / 0A_1A_1 / 1A_1$

$A_2 \rightarrow 0A_1A' / 1A' / 0A_1 / 1$

→ In the above all production are G.N.F